

Project Planning: Universal Language Learning Shortcut Tool

1. Core Functionalities (The "What")

To appeal to a broad audience, the tool should act as a modular pipeline: **Capture -> Process -> Output**.

- **Smart Capture:** Read text from the primary selection (highlighted) or clipboard, handling both Wayland (`wl-clipboard`) and X11 (`xclip`) seamlessly.
- **Quick Translation / Dictionary:** Trigger a popup or notification with the translation and dictionary definition of the captured text.
- **1-Click Flashcard Generation (Basic & Cloze):** Send formatted cards directly to Anki via AnkiConnect.
- **Dynamic Context Linking:** Automatically generate and attach URLs (like Reverso, Wiktionary, Jisho for Japanese) based on the target language.
- **Romanization / Phonetics (Optional Module):** Auto-generate pronunciation guides based on the language (Pinyin for Mandarin, Romaji/Furigana for Japanese, IPA for French/English).
- **Text-to-Speech (TTS):** (Stretch Goal) Download audio for the word/sentence and attach it to the Anki card automatically.

2. Configuration Strategy (Out-of-the-Box vs. Custom)

The secret to a successful open-source Linux tool is a zero-configuration "Happy Path" for beginners, with deep `.config` customization for power users.

What should work "Out of the Box" (Zero Config)

- **Default Language:** English to/from the system's default locale.
- **Basic Translations:** Using a free, key-less provider like `translate-shell` (which uses Google/Bing/Yandex under the hood).
- **Notifications:** Standard system notifications (via `libnotify` / `notify-send`).
- **Anki Deck & Model Generation:** If Anki is open and AnkiConnect is installed, your script should ping Anki. If the target Deck (e.g., "Quick Captures") or Model (e.g., "Auto-Language-Card") doesn't exist, **your script should automatically create them via AnkiConnect** with a standard format. This saves the user from setting up Anki fields manually.

What requires user configuration (`~/ .config/appname/config.toml`)

Users will need a config file (TOML or YAML are best for readability) to customize their workflow.

- **Language Pair:** Source language (e.g., `en`) and Target language (e.g., `zh` , `es` , `ja`).
- **Anki Mapping:**
 - Custom Deck Name.
 - Custom Model Name.
 - Field Mapping (e.g., `Field 1 = Original` , `Field 2 = Translation` , `Field 3 =`

Phonetics).

- **Context URLs:** A list of template URLs. E.g., `reverso = "https://context.reverso.net/translation/spanish-english/{word}"`
- **Translation Engine:** Option to switch from `translate-shell` to DeepL API, OpenAI API, or local Ollama instances for advanced users.
- **Display Preferences:** Choose between `notify-send` or a GTK popup (like `zenity / yad`) for longer texts.

3. Implementation Roadmap

Phase 1: Architecture & Language Choice

- **Action:** Move away from raw Bash. While Bash is great, handling JSON (for AnkiConnect) and parsing TOML/YAML config files is messy in Bash.
- **Recommendation:** Rewrite the core logic in **Python, Go, or Rust**.
 - *Python* is incredibly easy for text processing, JSON, and APIs.
 - *Go* compiles to a single, fast binary with zero dependencies (great for Linux packaging).
- **Action:** Define the CLI structure. The user shouldn't launch a GUI; they should map CLI commands to their window manager (e.g., `toolname --translate`, `toolname --cloze`).

Phase 2: Core Engine & Modularity

- Build the clipboard detection (auto-detect Wayland vs X11).
- Build the Translation module.
- Build the AnkiConnect API wrapper.
- *Crucial Step:* Isolate the "Pinyin" logic. Create a phonetics module that checks the target language. If language is Chinese, fetch Pinyin. If Spanish, skip it or fetch IPA.

Phase 3: Configuration & First-Run Wizard

- Implement a parser that reads `~/.config/appname/config.toml`.
- Create a CLI initialization wizard. When the user types `toolname --setup`, it asks:
 1. "What language are you learning?"
 2. "Do you want to use Anki?"
 3. "Generate default Anki templates? (Y/N)"
- Save these preferences to the config file.

Phase 4: Packaging and Distribution

- Publish the source code on GitHub/GitLab.
- Write a comprehensive `README.md` showing how to bind the commands in Hyprland, i3, Sway, and standard desktop environments like GNOME/KDE.
- Create an AUR (Arch User Repository) package.

- Create a `.deb` package or a simple `make install` script for other distributions.

4. Key Challenge to Anticipate

Window Manager Agnosticism: You cannot package a "shortcut tool" that automatically installs shortcuts into the system, because Linux users use wildly different window managers (Hyprland, i3, KDE, GNOME). **The Solution:** Your tool must act as a pure CLI application. In your documentation, provide copy-paste snippets for users to put into their respective config files (e.g., "Add this to `hyprland.conf`: `bind = $mod, T, exec, yourtool --translate`").

